Modern Congestion Control Algorithms

Yannik Schreiter
COMSYS
RWTH
Aachen, Deutschland
Yannik.Schreiter@rwth-aachen.de

***Abstract*—Abstract**
***Index Terms*—congestion control**

I. Introduction

Congestion Control Algorithms are an important topic in nowadays network technology. They are a necessary tool to control the network flow and to ensure the best use of network resources. In the following we are going to give a the necessary background to understand Congestion Control Algorithms and corresponding measuring techniques in II. Then we will show some of the newest Congestion Control Algorithms in III. After this we will evaluate and compare them to each other considering the information they provide over their algorithms in IV.

II. Background

A. Congestion Control Fundamentals

Congestion control encompasses the distributed algorithms designed to manage the transmission rate of senders to prevent network congestion. While there are algorithms for various protocols, the primary focus of Congestion Control development remains on the Transmission Control Protocol (TCP), where the objective is to maximize throughput and minimize latency while maintaining zero packet loss. Foundational work in this field identifies that congestion occurs when the aggregate demand for resources exceeds the available capacity of the network, specifically the bandwidth of links, buffer space in memory, and the processing capacity of intermediate nodes [1]. A significant challenge remains that upgrades to hardware are often disproportionate; for example, increasing transmission speed without a corresponding increase in CPU speed or buffer size can lead to new bottlenecks. Consequently, robust congestion control is essential to manage these hardware limitations effectively and ensure fair resource allocation.

1) Performance and Fairness Metrics: To evaluate the efficancy of a congestion control algorithm, several quantitative indices are employed. A primary metric is Jain's Fairness Index, which assesses how equal different flows share bottleneck resources [2]. This index yields a value between 0 and 1, representing the percentage of users who receive a fair share of the available capacity. A value of 1 signifies that 100% of the users are receiving an identical and fair portion of the resources, which is the ideal state for a distributed network environment [2].

B. TCP Cubic

As the Internet transitioned toward high-speed, long-distance network paths, the performance of traditional TCP variants was challenged by large bandwidth-delay products (BDP). Standard TCP implementations, such as Reno, utilize a linear window growth function that increases by only one packet per round-trip time (RTT), a process that is far too slow to fully utilize modern high-capacity links [3]. To address this scalability issue, TCP Cubic introduces a cubic window growth function that is independent of RTT [3]. This algorithm increases the congestion window aggressively when it is far from the saturation point and slows its growth as it approaches the previous limit, thereby maintaining high stability while remaining fair to standard TCP flows. Like its predecessors, Cubic is a loss-based algorithm that identifies the bottleneck capacity by monitoring for packet loss events [3].

C. TCP Vegas

The authors of TCP Vegas proposed an alternative to the reactive nature of TCP Reno, aiming to increase throughput while significantly reducing packet losses [4]. They recognized that loss-based algorithms are inherently inefficient because they must cause congestion and packet loss to determine available bandwidth. To create a more proactive mechanism, Vegas utilizes three key techniques. First, it employs a new retransmission mechanism based on fine-grained RTT timestamps, allowing it to detect and resend lost packets much faster than the coarse-grained timeouts used in Reno [4]. Second, it introduces a congestion avoidance scheme that compares the expected throughput to the actual throughput. By monitoring the difference between these rates, Vegas can adjust its window to keep a constant, minimal number of packets in the router queues, effectively avoiding congestion before it occurs [4]. Finally, it modifies the slow-start phase to grow the window only every other RTT, which allows the algorithm to detect congestion and transition to linear growth before a loss event occurs [4].

D. FAST TCP

Building on the delay-based insights of TCP Vegas, FAST TCP was designed specifically for high-speed, long-distance networks with higher latency. The authors identified that standard TCP implementations suffer at large windows due to slow linear increases and the instability caused by binary congestion signals [5]. FAST TCP addresses these issues by

using queuing delay as its primary congestion measure, which provides more precise information about the network state than simple packet loss. The algorithm allows the congestion window to be adjusted flexibly based on how far the current state is from equilibrium, rather than relying on iterative linear increments [5]. By stabilizing the network below the point of overflow, FAST TCP achieves high utilization and weighted proportional fairness while eliminating the severe oscillations common in loss-based protocols [5].

III. Related Work

A. Bottleneck Bandwidth and Round-Trip propagation Time (BBR)

Bottleneck Bandwidth and Round-Trip propagation Time (BBR) was developed by Google researchers to address fundamental inefficiencies in traditional loss-based congestion control algorithms [6]. The authors argue that modern hardware advancements have made the long-standing practice of using packet loss as a measure for congestion increasingly problematic [6]. Specifically, in environments with large network buffers, loss-based algorithms like CUBIC tend to keep buffers full, leading to "bufferbloat" where delays are measured in seconds rather than milliseconds [6]. Conversely, in long-distance networks, packet loss may occur due to link errors rather than actual congestion. However, loss-based algorithms misinterpret these signals, causing them to reduce throughput unnecessarily [6].

BBR operates through a state machine consisting of a Startup phase and two repeating steady-state phases known as ProbeBW and ProbeRTT. The authors define the optimal operating point as having a delivery rate equal to the bottleneck bandwidth ($BtlBw$) and a total amount of inflight data equal to the pipe's Bandwidth-Delay Product (BDP) [6]. During the Startup phase, BBR rapidly discovers the bottleneck's limits by increasing its sending rate by a pacing gain of $2/\ln 2 \approx 2.885$ every round trip until the delivery rate plateaus [6]. While this ensures the bottleneck is discovered in $\log_2 BDP$ rounds, it may temporarily fill the queue with up to $2BDP$ of excess data before transitioning into a Drain phase to empty the buffer [6], [7].

In steady-state operation, the algorithm primarily cycles through the ProbeBW phase to measure and adapt to changes in available bandwidth. This phase uses an eight-step cycle where the algorithm typically increases its sending rate by a pacing gain of 1.25 for one RTT to probe for additional capacity, then immediately reduces it to a gain of 0.75 for one RTT to drain the queue created during the probe [7]. Finally, to refresh the minimum round-trip time estimate, BBR enters the ProbeRTT phase if a lower RTT has not been observed for 10 seconds [7]. During this phase, the algorithm abruptly limits inflight data to a minimal amount—typically 4 packets—for at least 200 ms to fully drain the bottleneck buffer and measure the true propagation delay of the path [6], [7].

1) Active Queue Debt Compensation (AQDC) BBR: BBR utilizes the ProbeBW phase to estimate available bandwidth by periodically increasing its pacing rate. However, this process often injects bursts of traffic that exceed the path's instantaneous capacity, creating temporary queues [8]. The authors of [8] identify a critical limitation: these transient queues may not be completely drained during the subsequent ProbeDown phase, leading to a residual queue buildup that accumulates over successive cycles. Over time, this results in an upward drift of RTT measurements and increased tail latency, which is particularly detrimental to delay-sensitive applications like video streaming.

Furthermore, the authors contend that BBR's ProbeRTT mechanism is rigid and "blind" to queue history, as it resets at fixed intervals regardless of the actual backlog [8]. To reconcile the conflict between fairness and low latency, AQDC-BBR explicitly models the "queue debt" (B_{total}) incurred during bandwidth probing. This debt is then compensated for by adjusting the target inflight data:

$$Inflight_{target} = BDP - B_{total} \tag{1}$$

By proportionally reducing the inflight data based on the tracked debt, AQDC-BBR drains accumulated queues and restores accurate minRTT tracking without the need for aggressive or heuristic rate cuts [8].

2) BBR-Extended State (ES): TCP BBR exhibits significant RTT unfairness when flows with heterogeneous RTTs share a bottleneck [9]. In deep-buffer environments, long-RTT flows tend to dominate bandwidth because they maintain larger inflight data volumes—proportional to their RTT—which allows them to occupy the bottleneck queue more aggressively [9]. This leads to a positive feedback loop: higher queue occupancy results in higher delivery rate reports, which further inflates a flow's estimated bandwidth and sending rate [9]. While BBR's ProbeRTT phase is intended to synchronize flows and provide a "fair starting point" by draining the queue, the authors found that it often fails because the algorithm lacks an explicit mechanism to correct accumulated queue imbalances upon exiting this state [9]. Consequently, prior imbalances are immediately reintroduced and amplified, a phenomenon the authors describe as the "inaction" of BBR [9].

To resolve these issues, the authors introduced BBR-ES, which modifies the state machine by inserting a short stabilization phase, BBR-NEW-STATE, between ProbeRTT and ProbeBW [9]. This new state gradually ramps up the sending rate in three sub-phases based on per-flow RTT trends, allowing for a fairer recovery and the repair of inflight imbalances before returning to steady probing [9]. Additionally, BBR-ES includes a Trend-Driven Transition Mechanism that proactively triggers an early ProbeRTT only for flows exhibiting trends consistent with excessive queue contribution (specifically, a simultaneous increase in both bandwidth and RTT) [9]. This targeted approach prevents the "one-size-fits-all" penalty found in other BBR variants that often unnecessarily throttles low-impact flows [9].

B. Performance-oriented Congestion Control (PCC)

Performance-oriented Congestion Control (PCC) was developed to resolve a fundamental architectural flaw in the

TCP family known as "hardwired mapping," where specific packet-level events are pre-programmed to trigger fixed control responses [10]. This rigid structure forces TCP to rely on unreliable assumptions about the network, such as the belief that all packet loss indicates congestion, which leads to severe performance degradation on modern links where loss may be random or caused by shallow buffers [10]. Because traditional protocols carry out these harmful actions without observing their actual impact on performance, they often fail to achieve optimal throughput in diverse and complex real-world environments [10].

The authors solved this by re-architecting congestion control to rely on empirical evidence gathered through continuous "micro-experiments" rather than predetermined responses [10]. In this new architecture, the sender divides time into monitor intervals to test different sending rates and observes the resulting packet-level events [10]. These events are aggregated into a numerical utility function that reflects specific performance objectives, such as maximizing throughput while minimizing loss. By employing an online learning algorithm similar to gradient ascent, PCC compares the utility of different rates and consistently moves in the direction that empirically produces the best performance results [10]. This approach allows the protocol to adapt to complex network conditions without making prior assumptions, and the authors further improved decision-making accuracy under noisy conditions by utilizing randomized controlled trials (RCTs) to confirm the causal connection between a rate change and its observed utility [10].

C. Copa

Similar to the motivation behind BBR, the authors of Copa observed that traditional loss-based congestion control algorithms often cause significant "bufferbloat" at the network bottleneck [11]. This excessive queuing delay severely degrades the performance of interactive applications that require low latency [11]. The authors argued that existing delay-based algorithms, such as TCP Vegas or FAST TCP, are insufficient for modern internet conditions [11]. These protocols are frequently misled by network jitter or ACK compression and tend to be starved when competing against aggressive loss-based flows like TCP Cubic [11]. Furthermore, while newer "objective-oriented" or machine-generated algorithms like PCC may offer high performance, the authors noted they are often "black boxes" that are difficult for humans to reason about or tune [11].

To address these limitations, Copa is built upon three core design pillars [11]. First, it establishes a Target Rate ($\lambda = 1/\delta d_q$) derived from a Markovian packet arrival model [11]. In this formula, d_q is the measured queuing delay, which the sender calculates on every ACK as the difference between the recent standing RTT ($RTT_{standing}$) and the long-term minimum RTT (RTT_{min}) [11]. Second, it employs a Window Update Rule that incrementally adjusts the congestion window to converge rapidly toward this target rate, ensuring fairness even amidst high flow churn [11]. Third, it incorporates a Competitive Mode that utilizes an additive-increase/multiplicative-decrease logic on the δ parameter to allow Copa to detect and compete fairly with buffer-filling loss-based flows [11]. By leveraging the framework of Network Utility Maximization (NUM), Copa aims to maximize the utility function $U = \log \lambda - \delta \log d$, where λ is throughput and d is delay [11].

1) Copa+: The authors of Copa+ identified several critical vulnerabilities in the original Copa algorithm through mathematical analysis [12]. They demonstrated mathematically that Copa often fails to periodically clear the bottleneck buffer, which is a necessary step for the algorithm to accurately measure the $RTT_{standing}$ [12]. When the bottleneck buffer remains occupied, the minimum RTT observed by the sender is higher than the actual propagation delay [12]. This measurement error leads to a flawed estimation of the target rate and can cause the algorithm to mistakenly enter its Competitive Mode [12]. Although this mode is intended only for coexistence with loss-based flows like TCP Cubic, its accidental activation in the absence of such flows causes Copa to unnecessarily increase its congestion window [12]. This behavior results in persistent buffer occupancy, significantly increased queuing delays, and reduced fairness among competing flows [12].

To address these issues, the authors introduced Copa+, which incorporates a parameter adaptation mechanism and a refined mode-switching logic [12]. They observed that the original algorithm relies on a single "hard-wired" parameter, δ^{-1}, to determine both the user's performance preference and the granularity of window adjustments [12]. To break this dependency, Copa+ introduces a new parameter, θ, which serves as a rate adjustment factor [12]. This allows the algorithm to decouple the target queuing delay from the aggressiveness of the window updates [12]. Copa+ utilizes a cubic-based function to adaptively search for an optimal θ value [12]. specifically, it increases the adjustment granularity when it detects that the buffer is not being cleared, thereby forcing the queue to empty and allowing for a correct $RTT_{standing}$ measurement [12]. Once the buffer is cleared, the algorithm reduces the granularity to maintain stability [12]. Furthermore, the authors optimized the criteria for entering Competitive Mode by utilizing the algorithm's inherent oscillation cycles [12]. By tracking the direction changes in queuing delay, Copa+ can more accurately distinguish between genuine competition with loss-based flows and simple measurement noise, effectively preventing accidental mode activation and maintaining low latency across diverse network environments [12].

2) Copa-Dynamic (Copa-D): The authors of Copa-D focus their research on the unique challenges of highly volatile mobile networks, such as 4G and 5G [13]. Similar to the findings in Copa+ [12], they identify a fundamental performance degradation in the original Copa algorithm when deployed in unstable environments [13]. Specifically, they observe that Copa frequently fails to maintain its theoretical "empty queue" state [13]. This leads to standing queues at the bottleneck, which distorts RTT measurements and prevents the algorithm

from accurately sensing network congestion [13].

While acknowledging the approach taken by Copa+ (which introduces an additional parameter to the rate control equation) the authors of Copa-D propose an alternative strategy [13]. Rather than altering the underlying control structure, they focus on the core sensitivity parameter, δ [13]. In the original Copa implementation, δ is typically treated as a static constant, which the authors argue is the primary cause of its inability to adapt to cellular fluctuations [13]. Copa-D introduces a mechanism to make δ dynamic, automatically tuning it based on the observed bandwidth and a specific delay target [13]. This ensures that the queue is actively drained, effectively enforcing Copa's original design goal of near-zero queuing delay even within the unpredictable conditions of a mobile network [13].

D. PowerTCP

The authors of PowerTCP focus primarily on the unique demands of datacenter networks [14]. They argue that most existing congestion control algorithms suffer from a fundamental trade-off: they are either unstable or unable to react rapidly to network fluctuations [14]. "Voltage-based" algorithms, which react to absolute states like queue length or delay, tend to be stable but are often too slow to respond to the bursty traffic patterns typical of datacenters [14]. These schemes often fail to utilize available bandwidth quickly, especially in dynamic environments [14]. Conversely, "current-based" algorithms—those reacting to gradients or changes in the network—can respond quickly but often lack a unique equilibrium point, causing them to oscillate and become inherently unstable [14].

Furthermore, the authors anticipate a future where datacenter switches feature increasingly shallow buffers, significantly smaller than those found in traditional internet infrastructure [14]. They also highlight a common datacenter traffic characteristic: while the majority of flows are short-lived and sensitive to latency, the bulk of the actual data volume is generated by a small number of long-lived flows requiring high throughput [14]. Additionally, the emergence of reconfigurable datacenter networks (RDCNs) introduces rapid fluctuations in available bandwidth as optical circuits are re-established [14]. Given these evolving circumstances, the authors conclude that a hybrid congestion control approach—integrating both "voltage" and "current" signals—is necessary to maintain stability while achieving near-instantaneous reaction times [14].

PowerTCP addresses the limitations of existing datacenter congestion control by introducing the notion of network "power"—the product of absolute state (voltage) and its rate of change (current) [14]. The authors implement this via a window update law:

$$w_i(t) \leftarrow \gamma \cdot (w_i(t-\theta(t)) \cdot \frac{e}{f(t)} + \beta) + (1-\gamma) \cdot w_i(t) \quad (2)$$

In this equation, $w_i(t-\theta(t))$ represents the window size at the time of packet transmission, providing a historical reference for the update [14]. The term $e/f(t)$ acts as a power-based multiplicative factor, enabling rapid rate adjustments [14]. Stability and equilibrium are maintained through the additive increase parameter β and the exponential weighted moving average smoothing factor γ, which balances the system's responsiveness against network noise [14].

To gather the necessary network metadata, PowerTCP leverages In-band Network Telemetry (INT) [14]. Modern switches continuously monitor their internal state [14]. INT enables them to embed this real-time telemetry—such as queue lengths and timestamps—directly into the headers of passing TCP packets [14]. Upon arrival, the receiver extracts this telemetry and reflects it back to the sender within an ACK packet. This allows the sender to adjust its transmission rate based on the precise, current state of the network fabric [14].

E. Christian's Congestion Control Code - C4

The developers of C4 outlined three primary objectives for their algorithm [15]. First, the design prioritizes high performance in real-time applications, specifically video streaming [15]. Second, the algorithm is tailored for challenging access networks [15]. The developers specifically cite reports of degraded video quality in high-density Wi-Fi environments, such as buildings with many concurrent users [15]. This focus on unstable connectivity significantly influenced key design decisions [15]. Finally, the developers emphasized architectural simplicity and intended for the algorithm to be deployed primarily in conjunction with the QUIC protocol [15].

To understand how C4 operates, it is best to view it as a state-based controller that shifts through four distinct phases based on network feedback [16]. The algorithm begins in the Initial state, which functions similarly to a traditional "slow start" by rapidly increasing the data rate to identify the network's capacity [16]. Once the capacity is found, it transitions into the Cruising state. In this steady phase, the algorithm maintains a calculated Nominal Data Rate, which serves as its baseline for stable transmission [16]. To ensure that it is not leaving any available bandwidth unused, C4 will occasionally enter the Pushing state, where it carefully probes for higher capacity [16]. If the network becomes overloaded, triggered by packet loss or high delay, the algorithm moves into the Recovery state to stabilize the flow and redefine its baseline variables [16].

Central to C4's logic is the maintenance of "nominal" variables, specifically the Nominal Data Rate and the Nominal Max RTT [16]. By tracking these long-term metrics, the algorithm develops a memory of what the network looks like when it is healthy [16]. This is particularly important for its Jitter Tolerance mechanism [16]. In crowded Wi-Fi environments, temporary interference can cause spikes in delay that look like congestion but are actually just transient wireless noise [16]. C4 uses its nominal variables to set a minimum congestion window, allowing it to "ride through" these brief spikes without unnecessarily slashing its speed, which prevents video streams from stuttering [16].

Furthermore, C4 employs a unique Sensitivity Curve to manage fairness across different data flows [16]. This mechanism ensures that a flow's reaction to congestion is propor-

tional to its size [16]. Essentially, flows that consume a large amount of bandwidth are programmed to be more sensitive to congestion signals than smaller, time-critical streams [16]. By combining multiple signals—including traditional packet loss, RTT fluctuations, and Explicit Congestion Notification (ECN)—C4 creates a multi-layered defense against network instability [16]. This comprehensive approach makes it highly effective for Media over QUIC, where the primary goal is to maintain a high-quality, real-time video experience even under challenging network conditions [16].

IV. Evaluations

A. What is the goal of Congestion Control?

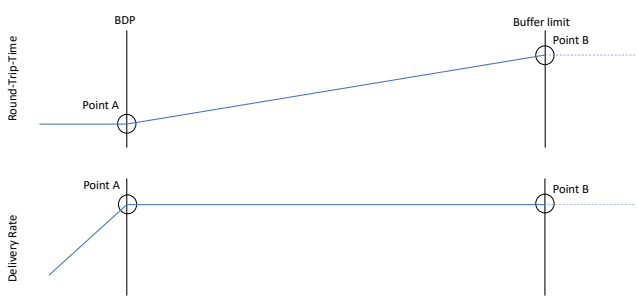

Fig. 1. Diagram that shows where different operating points for Congestion COntrol ALgorithms are. Based on [6], [7]

The fundamental goal of a Congestion Control Algorithm is to maximize throughput while maintaining a low delay. To determine the ideal performance of such an algorithm, it is necessary to identify the optimal operating point by observing how network dynamics shift as a sender increases its transmission rate. As illustrated in 1, the relationship between the sending rate, the delivery rate, and the RTT can be visualized through two primary diagrams. The top diagram tracks the RTT, while the bottom diagram displays the delivery rate, highlighting two critical thresholds labeled Point A and Point B.

Point A represents the moment the delivery rate equals the BDP, indicating that the bottleneck link is fully utilized. Beyond this point, the delivery rate plateaus because the bottleneck has reached its maximum physical capacity. If the sender continues to increase its rate past Point A, the excess packets are forced to accumulate in the bottleneck's buffer, creating a queue. This accumulation causes a linear increase in RTT, as seen in the upper diagram of 1, because packets must wait longer in the queue before being processed. Eventually, if the rate increases further, the system reaches Point B, which signifies the buffer limit. At this stage, the bottleneck can no longer accept new packets, leading to packet drops and severe congestion.

In addition to these performance metrics, another critical benchmark for evaluating a congestion control algorithm is its fairness toward other flows competing for resources at the bottleneck. While the behavior shown in 1 suggests that the optimal operating point for an individual sender is exactly at Point A, a high-quality algorithm must also ensure an equitable distribution of bandwidth. To quantify this behavior, one of the most widely utilized metrics for measuring the quality of resource distribution is Jain's Fairness Index. Therefore, an ideal congestion control algorithm should not only target maximum efficiency but also maintain a harmonious coexistence with concurrent flows to prevent starvation or unfair resource dominance.

Now we will take every mentioned algorithm from III and start to compare them to each other based on the mentioned metrics. First there is BBR for whose performance and quality the authors of [7] did a study. They tested BBR first with a different amount of flows of itself and then with TCP Cubic flows. The results show that multiple BBR flows will overestimate the available bandwidth which will result in congestion at the bottleneck. In scenarios with two BBR flows, the total BDP equals $2 * BDP$ of the actual BDP. In case the bottlenck has a huge buffer, it can handle the flows. Then the point where these BBR flows are operating is not at point B, but also not at the perfect point A. However, if the buffer of the bottleneck is small, the BBR flows are far more behind point B, which is a huge problem and will lead to a high packet loss.

Furthermore, the authors of [7] proved with experiements that if BBR competes with TCP Cubic flows will have a lot of problems. In case the bottleneck has a large buffer, TCP Cubic will fill this buffer and makes it hard for BBR to estimate the right RTT. In contrary, if the buffer is small, then BBR will surpress TCP CUbic, because TCP Cubic will interpret packet loss, that is induced by BBR as a congestion and send much less data. Considering that there are still a lot of TCP CUbic flows from not yet updated systems, BBR would most likely behave unfair in the internet.

B. Interprotocol fairness

C. RTT Fairness

D. Troughtput

E. Latency/Jitter

F. Jains Fairness Index

V. Summary

VI. Outlook

A. Future Problems

Which Problems new Congestion Control Algorithms have to face. Which standards do they have to fullfill.

B. Which direction it is going

References

FAST TCP [5]
COPA [11]
BBR Evaluation [7]
BBR Original [6]
TCP Cubic [3]
TCP Vegas [4]
PCC [10]
Raj Jain: Reasons for Congestion Control [1]
Jains Index [2]

References

[1] R. Jain, "Congestion control in computer networks: issues and trends," *IEEE Network*, vol. 4, no. 3, pp. 24–30, 1990.

[2] R. K. Jain, D.-M. W. Chiu, and W. R. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer system," *ACM Transactions on Computer Systems*, September 1984.

[3] S. Ha, I. Rhee, and L. Xu, "Cubic: A newtcp-friendly high-speed tcp variant," Princton Education.

[4] L. S. Brakmo and L. L. Peterson, "Tcp vegas: End to end congestion avoidance on a global internet," *IEEE SELECTED AREAS IN COMMUNICATIONS*, vol. 13, no. 8, October 1995.

[5] D. X. Wei, C. Jin, S. H. Low, and S. Hegde, "Fast tcp: Motivation, architecture, algorithms, performance," *IEEE/ACM Transactions on Networking*, vol. 14, no. 6, pp. 1246–1259, 2006.

[6] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobsen, "Bbr: Congestion-based congestion control," *ACM Queue: Measuring bottleneck bandwidth and round-trip propagation time.*, vol. 60, no. 2, February 2017.

[7] M. Hock, R. Bless, and M. Zitterbart, "Experimental evaluation of bbr congestion control," in *2017 IEEE 25th International Conference on Network Protocols (ICNP)*, 2017, pp. 1–10.

[8] Z. Li, J. Zhang, Y. Wang, P. Guo, and J. Jiang, "Aqdc-bbr: An active queue debt compensation mechanism over bbr for reducing tail latency in video streaming," in *2025 Thirteenth International Conference on Advanced Cloud and Big Data (CBD)*, 2025, pp. 56–61.

[9] Z. Han and G. Hasegawa, "Bbr-es: An extended-state optimization for bbr congestion control," *IEEE Transactions on Network and Service Management*, vol. PP, pp. 1–1, 01 2026.

[10] M. Dong, Q. Li, D. Zarchy, P. B. Godfrey, and M. Schapira, "Pcc: Re-architecting congestion control for consistent high performance," Paper in 12th USENIX Symposium on Networked Systems Design and Implementation, May 2015.

[11] V. Arun and H. Balakrishnan, "Copa: Practical delay-based congestion control for the internet," Paper in 15th USENIX Symposium on Networked Systems Design and Implementation, April 2018.

[12] W. Jiang, H. Li, Z. Liu, J. Wu, J. Huang, D. Shan, and J. Wang, "Copa+: Analysis and improvement of the delay-based congestion control algorithm copa," in *IEEE INFOCOM 2022 - IEEE Conference on Computer Communications*, 2022, pp. 920–929.

[13] H. Haile, K.-J. Grinnemo, S. Ferlin, P. Hurtig, and A. Brunström, "Copa-d: Delay consistent copa for dynamic cellular networks," in *2023 Joint European Conference on Networks and Communications and 6G Summit, EuCNC/6G Summit 2023, Gothenburg, Sweden, June 6-9, 2023*. IEEE, 2023, pp. 508–513. [Online]. Available: https://doi.org/10.1109/EuCNC/6GSummit58263.2023.10188233

[14] V. Addanki, O. Michel, and S. Schmid, "Powertcp: Pushing the performance limits of datacenter networks," Paper in 19th USENIX Symposium on Networked Systems Design and Implementation, April 2022.

[15] C. Huitema. (2025, November) Introducing c4, christian's congestion control code. Blog post. [Online]. Available: https://www.privateoctopus.com/2025/11/05/c4-introduction.html

[16] C. Huitema, S. Nandakumar, and C. F. Jennings, "Design of Christian's Congestion Control Code (C4)," Internet Engineering Task Force, Internet-Draft draft-huitema-ccwg-c4-design-03, Feb. 2026, work in Progress. [Online]. Available: https://datatracker.ietf.org/doc/draft-huitema-ccwg-c4-design/03/